



DAAN862: ANALYTICS PROGRAMMING IN PYTHON

Lesson 5: Data Wrangling with Pandas

Lesson 5.1: Hierarchical Indexing

Hierarchical indexing is a useful feature in pandas which enables multiple index levels on both axes (column and row). It provides a way to handle higher dimensional data with a lower dimensional format.

Please refer to Figure 5.1 for an example of a *Series* with a two lists as the index:



Fig 5.1

The output is a nice view of a Series with a two-level index. The outer and the inner levels are indicated in the Figure 5.1. The “gaps” in the outer-level index mean “use the index label above”.

To access the *index attribute* refer to Figure 5.2:

```
In [4]: data.index
Out[4]:
MultiIndex(levels=[['a', 'b', 'c', 'd'], [1, 2, 3]],
            labels=[[0, 0, 0, 1, 1, 2, 2, 3, 3], [0, 1, 2, 0, 2, 0, 1, 1, 2]])
```

Fig 5.2 (click to enlarge)

Figure 5.2 shows how to return Multiindex with both levels and labels the information.

If you are using hierarchical indexing, you can easily use an outer-level index to select subsets of the data as shown in Figure 5.3:

```
In [5]: data['b']
Out[5]:
1    -0.562908
3    -1.064038
dtype: float64

In [6]: data['b' : 'c']
Out[6]:
b 1    -0.562908
   3    -1.064038
c 1    -0.464033
   2     0.532145
dtype: float64

In [7]: data.loc[['b', 'd']]
Out[7]:
b 1    -0.562908
   3    -1.064038
d 2     0.962120
   3    -1.193371
dtype: float64
```

Fig 5.3 (click to enlarge)

To select subsets from an "inner" level refer to Figure 5.4:

```
In [8]: data.loc[:, 2]
Out[8]:
a    -0.910733
c     0.532145
d     0.962120
dtype: float64
```

Fig 5.4 (click to enlarge)

pandas provides a very convenient method `unstack` to reshape data into a pivot table. For example, you can rearrange the data created in Figure 5.5 into a DataFrame:

```
In [9]: data.unstack()
Out[9]:
```

	1	2	3
a	0.747121	-0.910733	-0.610212
b	-0.562908	NaN	-1.064038
c	-0.464033	0.532145	NaN
d	NaN	0.962120	-1.193371

Fig 5.5 (click to enlarge)

The DataFrame created will use the out-level index as row labels and the inner-level index as column labels. And it will fill NaNs to elements whose row-column label combinations don't exist in the original data.

stack is the inverse operation of `unstack`:

```
In [10]: data.unstack().stack()
Out[10]:
```

a	1	0.747121
---	---	----------

```

a 1 0.77121
   2 -0.910733
   3 -0.610212
b 1 -0.562908
   3 -1.064038
c 1 -0.464033
   2 0.532145
d 2 0.962120
   3 -1.193371
dtype: float64

```

Fig 5.6 (click to enlarge)

The two axes in the DataFrame can have a hierarchical index as shown in Figure 5.7:

```

In [11]: frame = pd.DataFrame(np.arange(12).reshape((4, 3)),
...:                          index=[['a', 'a', 'b', 'b'], [1, 2, 1, 2]],
...:                          columns=[['Ohio', 'Ohio', 'Colorado'],
...:                                   ['Green', 'Red', 'Green']])

In [12]: frame
Out[12]:

```

		Ohio	Colorado
		Green	Red
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

Fig 5.7

The highlighted regions in Figure 5. show the hierarchical indexes.

You can also give names to the hierarchical levels as shown in Figure 5.8:

```

In [13]: frame.index.names = ['key1', 'key2']    #set index level names as key1
and key2

```

```
In [14]: frame.columns.names = ['state', 'color'] # set column level names as
state and color

In [15]: frame
Out[15]:
```

		Ohio	Colorado
	color	Green	Red
	key1	key2	
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

Note: A red box highlights the column headers 'state' and 'color' in the original image, and a red arrow points to the 'Ohio' header.

Fig 5.8

As indicated by the arrow, the *frame* has level names now.

Similarly, you can select subsets of data by using column index:

```
In [16]: frame['Ohio']
Out[16]:
```

		Green	Red
	key1	key2	
a	1	0	1
	2	3	4
b	1	6	7
	2	9	10

Fig 5.9 (click to enlarge)

Reordering and Sorting Levels, Summary Statistics by Level, and Indexing with a DataFrames Columns

(click the tabs to learn more)

Rendering and Sorting Levels

Summary Statistics by Level

Indexing with a DataFrames Columns

Reordering and Sorting Levels

REORDERING AND SORTING LEVELS

Sometimes, you have to rearrange the order of axis levels or sort the data according to the values of a level.

The method ***swaplevel*** as show in Figure 5.10 can switch two levels:

```
In [17]: frame.swaplevel('key1', 'key2')
Out[17]:
```

	state	Ohio	Colorado
color	Green	Red	Green
key2	key1		
1	a	0	1
2	a	3	4
1	b	6	7
2	b	9	10

Fig 5.10 (click to enlarge)

sort index can sort the data for a level:

```
In [18]: frame.sort_index(level = 0)      # Sort frame by key1
Out[18]:
```

	state	Ohio	Colorado
color	Green	Red	Green
key1	key2		
a	1	0	1
a	2	3	4
b	1	6	7
b	2	9	10

```
In [19]: frame.sort_index(level = 1)      # Sort frame by key2
Out[19]:
```

	state	Ohio	Colorado
color	Green	Red	Green
key1	key2		
a	1	0	1
b	1	6	7
a	2	3	4
b	2	9	10

Fig 5.11

Summary Statistics by Level